



IDS – Project

Bug Tracking Information System

Viktor Čaloud

L'ubomír Gálik

March 15, 2026

Contents

1 Assignment	1
2 Data Model (ERD) and Use-Case Model	1

1 Assignment

Design an information system for reporting and managing software bugs and vulnerabilities. The system allows users to report bugs, their severity levels, and the modules in which they were found, in the form of tickets. A ticket may reference more than one bug, and the same bug can be reported independently by multiple users. A bug may optionally be classified as a vulnerability, in which case the potential severity of exploitation is also recorded. When bugs are reported, the system notifies the programmer responsible for the affected module; a programmer may be responsible for more than one module. The programmer then claims the ticket, changes its status to *In Progress*, and begins working on a fix in the form of a patch. A patch is characterised by its release date and must be approved by the programmer responsible for the module; modules may use different programming languages. A single patch may fix multiple bugs, resolve multiple tickets, and span several modules. Regular users may also submit patches, but such patches must undergo a more rigorous review before being deployed. In addition to the patch creation date, the system also records the date the patch was deployed to production. Each user and programmer is described by basic information (name, age, etc.) as well as the programming languages they are proficient in. Once bugs are fixed, users may be notified of the resolution and optionally rewarded with a monetary value based on the severity of the bug or vulnerability.

2 Data Model (ERD) and Use-Case Model

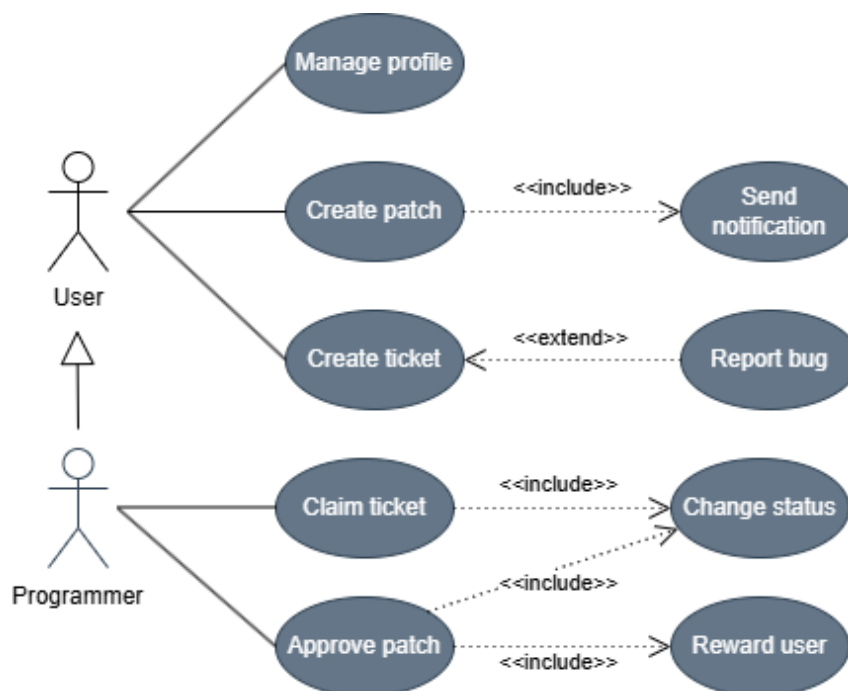


Figure 1: Use-case diagram

The data model represents an information system for bug reporting and management. As required by the assignment, the model includes a **generalisation/specialisation** relationship, where the programmer (**Programmer**) extends the base user entity (**User**). The process begins with users who report issues in the form of tickets (**Ticket**, 1:N relationship, creates). Since a ticket may describe multiple defects and the same defect may be reported independently by multiple users, the relationship between a ticket and a specific bug is of type M:N (**Bug**, contains). Bugs are always associated with specific system modules (**Module**, M:N relationship, in). The resolution process is managed by programmers who are responsible for modules (1:N relationship, is responsible for) and claim tickets for resolution (1:N relationship, solves). The physical code fix is represented by a patch (**Patch**). Any user may submit a patch proposal (1:N relationship, submits) with the aim of fixing one or more bugs (1:N relationship, fixes), but only an authorised programmer has the right to approve it for deployment (1:N relationship, approves). The architecture is completed by the programming language entity (**Programming Language**), which tracks the technologies used in modules (N:1 relationship, uses) and the language skills of individual users (M:N relationship, knows).

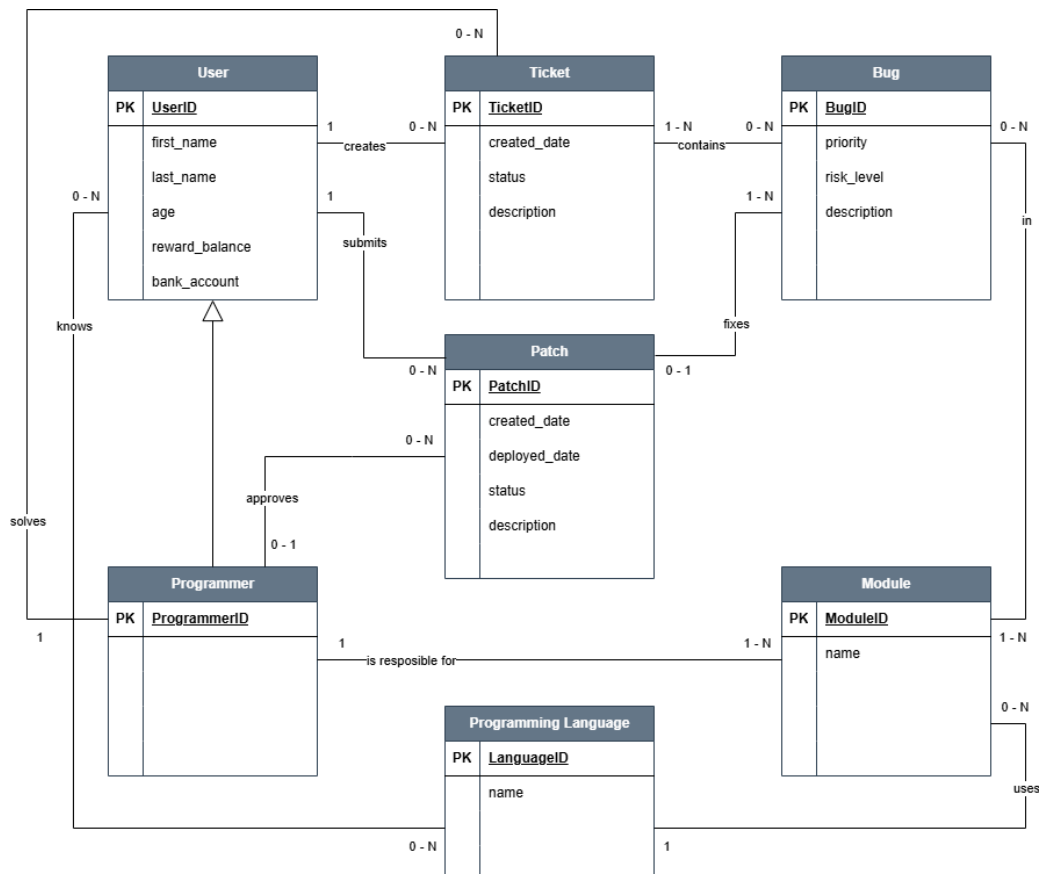


Figure 2: Entity-Relationship diagram